

AI for Coders

Delivered through the SDD – Dell Technologies Academy

حكومة الشارقة
دائرة الموارد البشرية
Government of Sharjah
Human Resources Department

Sharjah Digital Department
دائرة الشارقة الرقمية

DELL Technologies

COGNIT DX

حكومة الشارقة
دائرة الموارد البشرية
Government of Sharjah
Human Resources Department

البرنامج التطويري

AI For Coders (Cohort 1) - English

من 02/11/2026 الى 04/11/2026

مشرفة البرنامج : امنه الشامسي

تسجيل الحضور

NEXT

Now let us build GitHub

Three steps, and you will have asked Gemini your first grounded question. ----→ Download Data



https://github.com/Decoding-Data-Science/sharjah

01

Create your free key

AI Studio, then a Colab Secret.

02

Understand the data

Lab 1: the airport database and a dashboard.

03

Ask Gemini with context

Lab 3: model, system prompt, facts.

CORPORATE WORKSHOP · DAY 2 / 3

Generative AI and RAG

From a prompt in Gemini to an assistant that answers from your airport documents

Gemini + system prompt | Gradio app | RAG with LlamaIndex | Airport policy PDFs

Instructor: Mohammad Arshad Ahmed

WHERE WE ARE

The three-day journey

Day 1

Prompts and data

Airport data, prompt engineering, Gemini in code, system prompt and context.

Day 2 · today

Gen AI and RAG

Gemini recap, then RAG with LlamaIndex on the airport policy PDFs, and a Gradio app.

Day 3 · morning

Agentic AI

An agent that uses tools: your RAG and your SQL data.

Day 3 · afternoon

Team project

Each team builds and demos a simple project in Colab.

Today's system prompt and RAG become tomorrow's agent tools.

THE LEARNING METHOD

Understand first. Then use AI.

Five moves from foundations to a working team solution, and where Day 2 sits

01

CONCEPTS

Terms + big picture

Today: why RAG, chunks, embeddings, top-k

02

DOMAIN

Know the data

Today: the airport policy and procedure PDFs

03

DO IT

Follow each step

Today: the RAG notebook, step by step

04

AI-ASSISTED

Prompt with intent

Today: the system prompt and the rule prompt

05

TEAM CAPSTONE

Solve + showcase

Day 3: build and demo with your team

↻ **KAHOOT CHECKPOINTS** Quick recall and revision between stages

The better you understand the fundamentals, the better you can direct AI.

TODAY

Three parts, one story

Part 1 — Recap

Gemini, system prompt and context

Call Gemini from code, add a system prompt and evidence, and see why plain Gemini is not enough.

`Slides and live demo`

Part 2 — Hands-on

RAG with LlamaIndex and Gemini

Load two airport PDFs, chunk, embed, search top-k = 3, answer with sources, add a Gradio app.

`Day2_Simple_RAG_Gemini_LlamaIndex.ipynb`

Wrap-up — Quiz

Revise, then look ahead to Day 3

Kahoot checkpoint on chunking, embeddings and top-k. Then RAG becomes an agent tool.

`Kahoot + Day 3 preview`

All in Google Colab with your Gemini key in Secrets. You upload the two airport PDFs. Nothing to install on your laptop.

BEFORE WE START

Five-minute setup

1 Open Colab

colab.research.google.com, signed in with a Google account.

2 Add your key to Secrets

Key icon on the left > Add new secret > Name: GEMINI_API_KEY > Notebook access ON.

3 Get the two PDFs

GIA-POL-01 Passenger Service Standards Policy and GIA-SOP-06 Queue Escalation SOP. You upload them in Step 3.

4 Open the notebook

File > Upload notebook, then choose Day2_Simple_RAG_Gemini_LlamaIndex.

5 Run top to bottom

Shift + Enter on each cell. Errors: read the last line first.

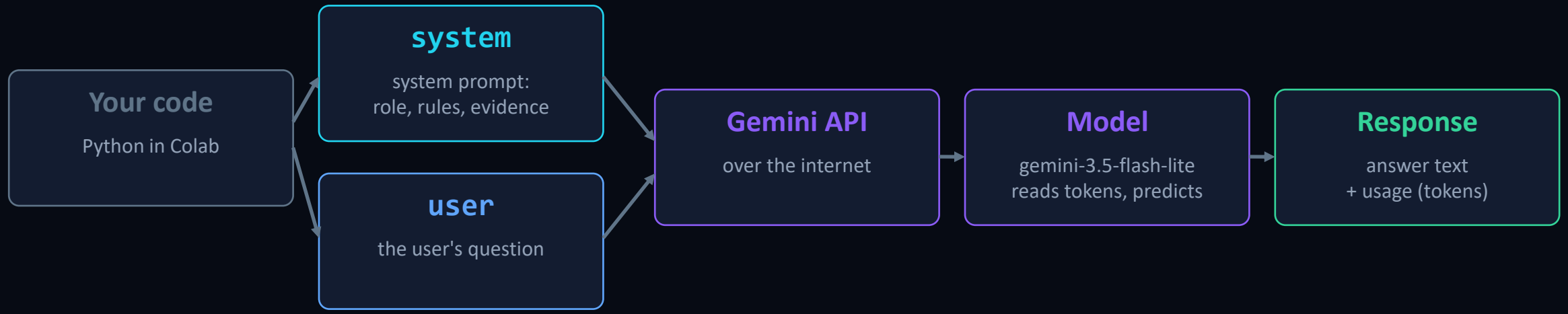
PART 1

Recap: talking to Gemini in code

Tokens, the system prompt and evidence: the Day 1 ideas that RAG builds on

HOW IT WORKS

One request, in code




```
messages = [
    ChatMessage(role="system", content=SYSTEM_PROMPT),   # the system prompt
    ChatMessage(role="user", content="What is the maximum average wait?"),
]
response = llm.chat(messages)
print(response.message.content)
```

Why tokens matter

Context window

Gemini models have large context windows (prompt + answer together), but never an infinite one.

Cost

You pay per token: input + output. Longer prompts and longer answers cost more.

Speed

More tokens in and out means slower answers. Short, focused prompts are faster.

2 PDFs ≈ 3 short pages

Paste everything into every prompt? Fine for 2 PDFs. Not for 2,000. RAG sends only the 3 most relevant chunks: cheaper, faster, and the model is not lost in a haystack.

SYSTEM PROMPT

The system prompt is the model's job description

`System prompt → system message`

ROLE You are the Airport Operations Assistant.

RULES Use ONLY the evidence. If it is not there, say "I could not find this in the documents".

FORMAT Cite the document. Keep the answer under 120 words.

EVIDENCE GIA-POL-01: average wait at departure immigration must not exceed 15 minutes. Terminal 1 needs 9 counters open at peak hours.

`User prompt → user message`

"What is the maximum average wait for departure immigration?"

Stays vs changes

The system prompt stays the same for every question. The input changes each time.

Day 1 prompt = Day 2 system prompt. Same skill, now in code.

Same model, same question, three results

1 · No evidence

Q: "What is the maximum average wait for departure immigration?"

Guesses a typical limit, or says it cannot know. It has never seen our policy.

2 · System prompt + evidence

Same question, with the policy line in the system prompt

"15 minutes, according to GIA-POL-01."

3 · Question outside the data

Q: "What is the daily parking tariff at the airport?"

"I could not find this in the documents." No invented tariff.

Ground it with evidence, and give it permission to say "I don't know".

PART 2

RAG: chat with your airport documents

Chunking, embeddings, vector stores, semantic search and top-k with LlamaIndex and Gemini

The model does not know your documents

The problem

Trained on public data up to a cut-off date. Never saw our Passenger Service Standards Policy or the Queue Escalation SOP. Ask about them and it guesses.

Option 1 · Paste everything

Every document in every prompt.
Expensive, slow, and the model loses the needle in the haystack. Does not scale past a few files.

Option 2 · RAG

Retrieve only the 3 most relevant pieces, then generate the answer from them, with sources a person can check.

RAG = Retrieval-Augmented Generation = first RETRIEVE the right pieces, then GENERATE the answer.

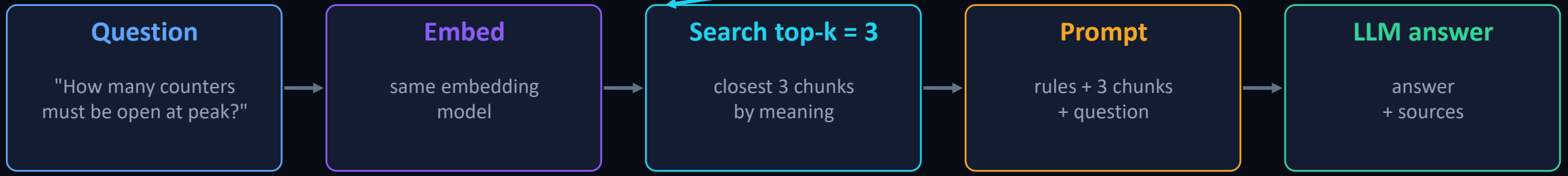
Two phases: build the index once, then ask many questions

PHASE 1 · INDEX (once)



search the vector store

PHASE 2 · QUERY (every question)



The model only ever sees the 3 retrieved chunks, never the whole library.

Cut documents into pieces the search can find

One document

Scope and purpose

Wait-time standards

Counter staffing

Escalation levels

Smart gates

chunk 1 · Scope and purpose

chunk 2 · Wait-time standards

chunk 3 · Counter staffing

chunk 4 · Escalation levels

chunk 5 · Smart gates

Illustration: topics in a policy. Overlap 10%

`chunk_size = 200`

precise, may lose context

most chunks

`chunk_size = 300`

our setting today

balanced

`chunk_size = 600`

more context, vaguer search

fewest chunks

Too small: pieces lose meaning. Too big: vague search and wasted tokens. We test 200, 300 and 600.

STEP 4 · CHUNK OVERLAP

Overlap keeps ideas that cross a border

Without overlap

...average wait must not exceed 15 minutes. Above 20

minutes, the duty manager takes action...

Chunk 1 and chunk 2 each hold half a rule. The search can miss it.

With overlap (30 tokens)

chunk 1: ...average wait must not exceed 15 minutes. **Above 20 minutes,**

30 / 300 = 10%

Typical: 10 to 20% of the chunk size.
LlamaIndex default: 1,024 / 200.

chunk 2: **Above 20 minutes,** the duty manager takes action...

amber = the overlap, repeated in both chunks. Chunk 2 now holds the complete rule.

Turning meaning into numbers

"The immigration queue is very long on Friday nights."

↓ gemini-embedding-001

```
[ 0.0123, -0.0441, 0.0870, 0.0036,
  -0.0219, 0.0512, ... ]

768 numbers = 1 vector
```

Similar meaning → vectors close together

No word "queue" or "immigration", but it lands next to those sentences.



STEP 6 · VECTOR STORE

Where the chunks and their vectors live

id	chunk text	vector (768 numbers)	metadata
1	"Average wait at departure immigration must not exceed 15 minutes..."	`[0.012, -0.044, 0.087, ...]`	file: GIA-POL-01
2	"At peak hours Terminal 1 must have 9 counters open..."	`[-0.031, 0.020, 0.064, ...]`	file: GIA-POL-01
3	"When the average wait goes above 20 minutes, the duty manager..."	`[0.008, -0.052, 0.071, ...]`	file: GIA-SOP-06
...	The notebook prints the real chunk count	`...`	...

Metadata = sources

The file name and page travel with every chunk, so every answer can cite its document.

Saved to disk

persist() writes small JSON files: embed once, reload later.
Production: a vector database (Pinecone, Chroma, pgvector).

STEP 7 · SEMANTIC SEARCH

How top_k = 3 works

"How many departure immigration counters must be open in Terminal 1 at peak hours?"

```
question vector
[0.021, -0.038, 0.079, ...]
```

cosine similarity:
1.0 = same meaning, 0 = unrelated

top_k = 3

0.71	**GIA-POL-01 · counters open at peak hours**
0.66	**GIA-POL-01 · installed counters, Terminal 1**
0.58	**GIA-SOP-06 · reserve counters at peak**
0.41	GIA-POL-01 · smart gates, Terminal 1
0.33	GIA-SOP-06 · escalation above 20 minutes
0.27	GIA-POL-01 · scope and purpose

Illustrative scores. Notebook Step 6 prints the real ones.

Only the top 3 chunks go to the LLM. k too small: missed evidence. k too big: noise and cost.

Search by meaning, not by matching words

Q: "Why do passengers wait so long at passport control?"

Keyword search (Ctrl+F)

Looks for the words "passport control".
The documents say "departure immigration".

0 results.

Semantic search (embeddings)

Compares meaning.
"passport control" ≈ "immigration".

Finds: the chunk on the departure immigration wait limit.

Notebook Step 4: three sentences, one comparison. Read the similarity scores yourself.

The whole RAG pipeline in a few lines

```
Settings.llm = GoogleGenAI(model="gemini-3.5-flash-lite")
Settings.embed_model = GoogleGenAIEmbedding(model_name="gemini-embedding-001")
```

1 **Settings: Gemini model and embeddings**

```
documents = SimpleDirectoryReader("/content/airport_rag").load_data()
```

2 **Load the two PDFs you uploaded**

```
nodes = SentenceSplitter(chunk_size=300,
chunk_overlap=30).get_nodes_from_documents(documents)
```

3 **Chunk**

```
index = VectorStoreIndex(nodes)
```

4 **Embed + vector store (then save)**

```
retriever = index.as_retriever(similarity_top_k=3)
```

5 **Semantic search, top 3**

```
query_engine = index.as_query_engine(similarity_top_k=3)
response = query_engine.query("What must the duty manager do above 20 minutes?")
```

6 **Retrieve + generate + sources**

STEP 10 · SETTINGS

Six settings that shape your RAG system

`Settings.llm`

gemini-3.5-flash-lite

Writes the answer from the chunks. Temperature, top_p and top_k are ignored by this model.

`Settings.embed_model`

gemini-embedding-001 · 768 dimensions

Question and chunks must use the SAME embedding model.

`chunk_size`

300 tokens

Smaller = more, precise chunks. Bigger = fewer, broader chunks.

`chunk_overlap`

30 tokens (10%)

Keeps sentences that cross a chunk border.

`similarity_top_k`

3

How many chunks reach the LLM. Try 1, 3 and 5 and compare.

`text_qa_template`

Answer only from the context

Cite the document. Say "I could not find this in the documents" when the answer is missing.

STEP 11 · THE APP

From a RAG engine to a chat window

1

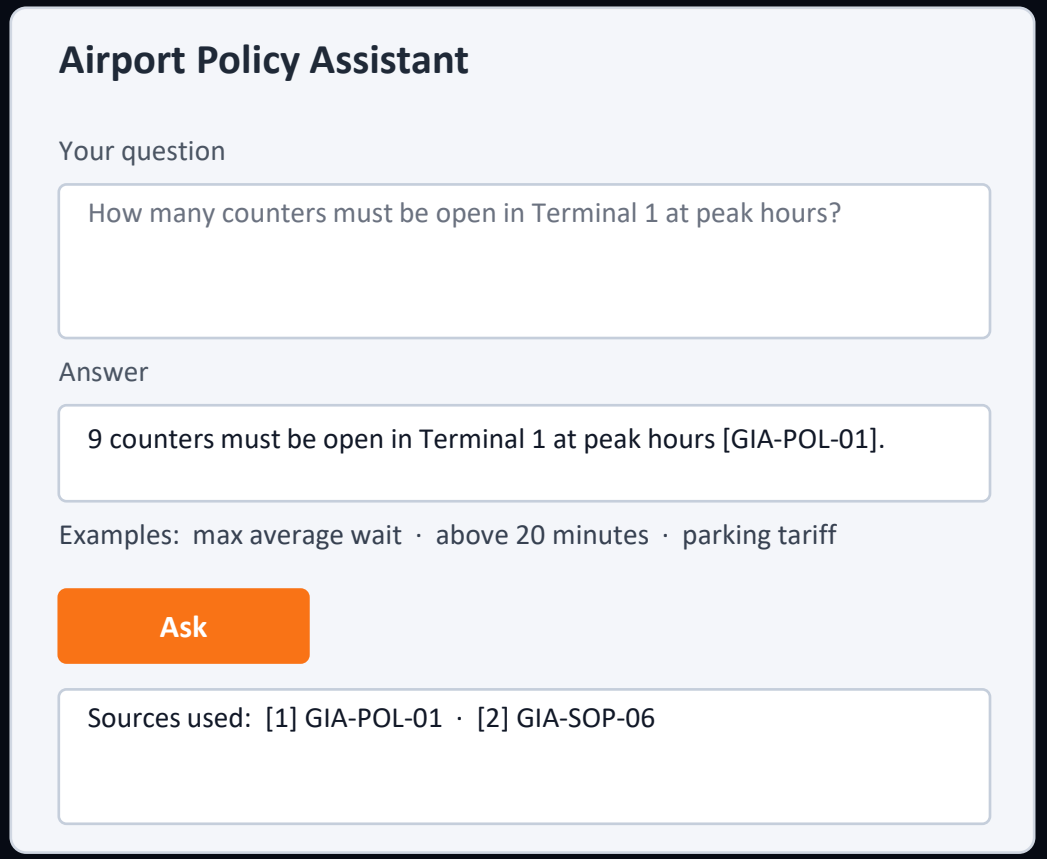
```
def answer_question(question):
    response = final_engine.query(question)
    return str(response), sources
```

2

```
with gr.Blocks() as demo:
    question_box = gr.Textbox(label="Your question")
    ask_button = gr.Button("Ask")
    answer_box = gr.Textbox(label="Answer")
    sources_box = gr.Textbox(label="Sources used")
    ask_button.click(answer_question, ...)
```

3

```
demo.launch(share=True)   # link appears under the cell
```



Ask the parking question with and without the rule prompt, and watch the answer change.

HANDS-ON LAB

RAG with LlamaIndex and Gemini · Day 2 notebook

1 **Steps 1 to 2 · Install and connect**

Install, key in Secrets, test Gemini and the embedding model.

2 **Step 3 · Load the PDFs**

Upload GIA-POL-01 and GIA-SOP-06. One document per page.

3 **Step 4 · Embeddings**

768 numbers. Same meaning, different words: compare the scores.

4 **Step 5 · Index and save**

Chunks of 200, embed, vector store. Open the saved files.

5 **Step 6 · Ask questions**

Top-k = 3: answer, chunks, scores. A two-document question, and one that is not there.

6 **Steps 7 to 9 · Tune and app**

Scoreboard for chunking and top-k. Lock 300 / 30 / 3, rule prompt, Gradio app.

Your turn: chunk_size 200 or 600 · top_k 5 · ask with "passport control" · add a third PDF.

When RAG goes wrong, and what to check

Wrong chunk retrieved

Print the top-k chunks first. Then tune the chunk size or top_k.

Answer not in the documents

Use a rule prompt: "I could not find this in the documents". Test with the parking tariff question.

Old and draft versions

Search cannot tell approved from superseded. Index only approved versions, or add status metadata and rules.

No sources, no trust

Always show the sources used. A person checks and decides.

WRAP-UP

What you built today, and where it goes tomorrow

System prompt + context

Your structured prompt in code, with evidence.

RAG with LlamaIndex

Chunk, embed, store, search top-k = 3, answer with sources.

Gradio chat app

Answer and sources in a window managers can use.

Day 3 · Agentic AI

Agent = LLM that chooses tools

RAG tool

documents

Data tool

airport database

Gemini

chooses the tool

Tonight: keep your Gemini key in Colab Secrets and your two airport PDFs. We reuse both tomorrow.

حكومة الشارقة
دائرة الموارد البشرية
Government of Sharjah
Human Resources Department

البرنامج التطويري

AI For Coders (Cohort 1) - English

تسجيل الانصراف

تقييم البرنامج